

Introduction to C

EPCC HPC Summer School 2026

William Lucas

EPCC



Contributors

In this material:

- The slides are created using $\text{\LaTeX}$ *Beamer* available at <https://ctan.org/pkg/beamer>.

License - Creative Commons BY-NC-SA-4.0¹

Non-Commercial you may not use the material for commercial purposes.

Shared-Alike if you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

Attribution you must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

¹You can find the full documentation about this license at:

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Prerequisites

The content presented in this material assumes that:

- You know how to **open a terminal** on your machine, and are familiar with **essential shell commands** such as moving through file hierarchies.

Nota bene

Format

This set of slides is designed to be read as a **stand-alone document** for self-study purpose, and thus contains a lot of text and explanations. It is **less suitable** to be delivered as a live presentation.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Structure

- The content presented in this topic is designed in small units that typically take **15 minutes of less**.
- Each unit is made of:
 - A few slides introducing a new technique or concept.
 - A short exercise illustrating this technique or concept.

Structure

- This approach using units makes sure you get to learn **and practice** every technique in turn.
- At the end, you will have one final exercise that will make use of everything you will have learned.
- Do not worry, they are guidelines and directions throughout the entire coding refresher.

Getting ready

- The first step will be to acquire the exercises.
- You can find them in the archive available on Learn, next to the slides.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Declaration

In the C programming language:

- Variables cannot be used before being declared.
- Variables must be assigned a data type on declaration.
- A variable cannot change its data type.
- Like all statements in C, variable declarations must end with a semi-colon ';'.

Structure of a declaration:

```
1 type_of_variable name_of_variable;
```

Variable names

- Variable names are *case-sensitive*: lowercase and uppercase letters are not treated as identical.
 - Variables `'my_var'` and `'MY_VAR'` are distinct.
- The use of **digits** in variable names is allowed **except** for the first character.
 - Valid: `my_var_1`
 - Invalid: `1_my_var`
- Variable names can contain underscores `'_'`, but **no other** special characters.
 - Valid: `my_var_123`
 - Invalid: `my-var-123`

Basic data types

`int` A standard signed integer

`unsigned int` A standard unsigned integer

`float` A floating-point number

`double` A double-precision floating-point number

`bool`² A boolean

²Requires the `stdbool.h` library

Examples of declarations

```
1 int a;  
2 unsigned int b;  
3 float c;  
4 double d;  
5 bool e;
```

Assignment

- Variable assignments use the operator '='.
- Variables can be assigned a value upon declaration.
- Variable declarations and variable assignments, like all statements in C, must end with a semi-colon ';'.

Examples of declarations and assignments

```
1 int a = -123; // Declaration + assignment
2 a = 456; // Assignment
3
4 float b = 0.456;
5 b = 0.789;
6
7 double c = 0.456;
8 c = 0.789;
9
10 bool d = true;
11 d = false;
```


Time to practise: 1.Variables

In this exercise, you will:

- Declare variables, using different types.
- Initialise variables.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

What is it?

- In C, like other programming languages, you have operators that allow you to have different interactions with your variables.
- The most basic ones are **arithmetic operators**, which allow you to perform arithmetic operations on your variables.

Arithmetic operators

Addition

```
1 a = b + c
```

Subtraction

```
1 a = b - c
```

Multiplication

```
1 a = b * c
```

Division

```
1 a = b / c
```

Modulo

```
1 a = b % c
```

Concatenated

- Sometimes, we want to store the result of an arithmetic operation in one of the variables part of the said operation. For example:

```
1 a = a * 20  
2 b = b - c
```

- For convenience, in C, there are *compound assignment operators*, which embed the arithmetic operation in the assignment.

Semantics

Compound assignment operators are syntactic sugar, however, they bring no additional functionality.

Compound assignment operators

■ Addition +=

```
1 a += b; // Equivalent to a = a + b;
```

■ Subtraction -=

```
1 a -= b; // Equivalent to a = a - b;
```

■ Product *=

```
1 a *= b; // Equivalent to a = a * b;
```

■ Division /=

```
1 a /= b; // Equivalent to a = a / b;
```

■ Modulo %=

```
1 a %= b; // Equivalent to a = a % b;
```

Compound assignment operators

Another typical situation, in loops in particular, is to increment or decrement a variable. For instance:

```
1 a = a + 1; // Incrementing  
2 a = a - 1; // Decrementing
```

Each operation has its compound assignment operator:

■ Increment ++

```
1 a++; // Equivalent to a = a + 1;
```

■ Decrement --

```
1 a--; // Equivalent to a = a - 1;
```

Time to practise: 2.Operators

In this exercise, you will:

- Practice the assignment operator.
- Practice arithmetic operators.
- Practice compound assignment operators.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Conditional execution

- Sometimes, a certain part of the code is meant to be executed only if a given condition is satisfied.
- To this end, we use conditional statements or branches.

How it works

- Create a conditional statement using the `if` keyword.
- Provide a condition.
- Provide a code block to execute if the condition is met.
- Provide one or more alternative branch(es) using `else if` statements.
- Provide a `else` statement that will be executed if all other `if` and `else if` statements did not execute.
- A conditional statement can be made only of a single `if`, without any `else if` or `else` blocks.

Example

■ An `if` statement alone.

```
1 if(condition1)
2 {
3 }
```

■ An `if` statement with an `else` statement.

```
1 if(condition1)
2 {
3 }
4 else
5 {
6 }
```

Example

- An `if` statement with `else if` statements, and an `else` statement.

```
1  if(condition1)
2  {
3  }
4  else if(condition2)
5  {
6  }
7  else if(condition3)
8  {
9  }
10 else
11 {
12 }
```

Comparison operators

- In addition to assignment and arithmetic operators, there are *comparison operators*: they evaluate an expression and **return a boolean** value.
- Comparison operators are typically found in **conditional statements** (i.e.: `if` and `else if`).

Comparison operators

■ Equal to ==

```
1 if (a == b)
```

■ Not equal to !=

```
1 if (a != b)
```

Caution

Note the double '=' for the equal to symbol. If you use only a single '=', it becomes an assignment and, without going into details, will still somehow evaluate to a boolean eventually but certainly not the value you were expecting.

Comparison operators

■ Strictly lesser than <

```
1 if(a < b)
```

■ Lesser than or equal to <=

```
1 if(a <= b)
```

■ Strictly greater than >

```
1 if(a > b)
```

■ Greater than or equal to >=

```
1 if(a >= b)
```


Logical operators

To combine your conditions, you will need *logical operators*.

■ Logical AND &&

```
1 if(a==2 && b<3) // true if both a==2 and b<3
```

■ Logical OR ||

```
1 if(a==2 || b<3) // true if a==2, b<3 or both
```

■ Logical NOT !

```
1 if(!(a==2 && b<3)) // true if a!=2, b>=3 or both
```

Caution

Operators && and || are made of **two symbols**. Without going into details, using a single character will still compile but do something totally different than what you expect.

Time to practise: 3.Branching

In this exercise, you will:

- Create branches with “if”, “else if” and “else” blocks.
- Use comparison operators.
- Use logical operators.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

What is it?

- Loops are structure that allow to repeat the execution of a block for as long as a given condition holds.
- There are multiple loop structures in C, the two main ones are:
 - `while`
 - `for`

While loop

- The *while* loop takes a code block, which it will execute for as long as the condition given holds.

```
1 while(condition)
2 {
3     // Code to repeat.
4 }
```

A classic scenario

- A classic situation is a loop that executes for a given number of iterations.
- It always comes down to the following structure:

```
1 int max_iterations = 10;  
2 int current_iteration = 0;  
3 while(current_iteration < max_iterations)  
4 {  
5     // Code to repeat.  
6     current_iteration++;  
7 }
```

The `for` loop

- Another loop structure is available, as a syntactic convenience: the `for` loop.
- The `for` loop embeds “everything” in its structure, separated with semi-colon ‘;’:
 - Part A The declaration and/or initialisation of the iterator.
 - Part B The condition that will indicate whether to continue the loop.
 - Part C The modification to apply to the iterator at the end of every iteration.

```
1 for(<part A>; <part B>; <part C>)  
2 {  
3     // Code to repeat.  
4 }
```

The for loop

- You are free to declare the iterator before the loop, or inside it:

- Declaring the iterator before the loop

```
1 int i;  
2 for(i = 0; i < some_value; i++)  
3 {  
4     // Code to repeat.  
5 }
```

- Declaring the iterator in the loop

```
1 for(int i = 0; i < some_value; i++)  
2 {  
3     // Code to repeat.  
4 }
```


Loops execution flow

- During the execution of a loop, there are two ways with which you can interfere with the execution flow:
 - `continue`
 - `break`

Skipping an iteration

`continue` Interrupts the current iteration and goes directly to the condition check phase for the next iteration.

```
1 while(conditionA)
2 {
3     if(conditionB)
4     {
5         continue;
6     }
7     // Section not executed if conditionB holds
8 }
```

Execution flow - Break loop

break Interrupts the current iteration and exits the loop. If there are nested loops, it breaks out only of the innermost loop from which it was called.

```
1 while(conditionA)
2 {
3     if(conditionB)
4     {
5         break;
6     }
7 }
```

Time to practise: 4.Loops

In this exercise, you will:

- Create `for` and `while` loops.
- Use `continue` to skip iterations.
- Use `break` to interrupt the loop execution.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Printing

- The function `printf` allows you to print text in C.

```
1 printf(<formatted string>, <variables>);
```

- To inject a variable, the code depends on the variable type:

`%c` A character.

`%d` An integer.

`%f` A float or a double.

`%p` A pointer.

`\t` A tabulation.

`\n` A line break.

- You must put the variables in the order they are referred to in the format string.
- Requires `#include <stdio.h>` (stands for standard IO).

Example

■ Printing just text

```
1 printf("Some text, and no variables.\n");
```

■ Printing a single variable value

```
1 int a = 123;  
2 printf("Value of variable 'a' is %d.\n", a);
```

■ Printing multiple variables value

```
1 int a = 123;  
2 float b = 456.789;  
3 printf("Value of variable 'a' is %d, and the value  
   of variable 'b' is %f.\n", a, b);
```

Structure of a function

- The function `printf` works in an unusual way.³
- However, for the vast majority of functions, they rely on a classic structure.

```
1 return_type function_name(arg_1_type arg_1_name,  
2                           arg_2_type arg_2_name,  
3                           ...,  
4                           arg_n_type arg_n_name)  
5 {  
6     // Code  
7 }
```

³For curious students, see variadic functions.

Define your own - Arguments

- All arguments are **compulsory**; there are no optional arguments.
- Arguments cannot be named like in Python.
- Every argument must be accompanied with its type.
- The variables you receive are “passed by value”, in other words, they are copies. Any modification on them will not change the value of the original variable (you will see how to do this later on).
- Functions cannot be overloaded.

Define your own - About the return

- A function not returning anything is said to have a return type of `void`.
- A function either returns nothing or it returns one value. It cannot return multiple values.
- If a function returns something (i.e.: not `void`), then it must have a `return` keyword somewhere.

Do not forget the `return` keyword

... or “missing return statement from non-void function” will be the compiler’s way to remind it to you.

Examples of functions

```
1 int add_two_ints_together(int a, int b)
2 {
3     return a + b;
4 }
```

```
1 void print_int_value(int a)
2 {
3     printf("The int passed has value %d.\n", a);
4 }
```

Function declaration location

- **For now**, place your functions above the function `main`. If you put them below it, it will not work.
- **Later on**, you will learn how to properly structure a source code, and place your functions anywhere you want.

Time to practise: 5.Functions

In this exercise, you will:

- Create your own functions.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

What is it?

Definition

A *pointer* is a variable whose value is the **memory address** of another variable.

What is it?

- When a pointer points nowhere, we can set its value to `NULL`.
- Pointers have a type too: the type of the variable they point to.
- Pointers can be used to **pass variables by reference** to functions. In other words, making sure the function modifies the original instance of the variable, not a copy of it.

Pointer syntax

Pointers need some new syntax:

- Declare a pointer by placing a `*` symbol after the type it's going to point to.
- The reference operator `&` when placed before a variable name returns that variable's address in memory.
- The dereference operator `*` when placed before a pointer works the opposite way, by returning the value in the memory that is pointed to.

Remember that `*` has two different meanings when it comes to working with pointers.

Example

```
1  int a = 123;
2
3  // Declare a pointer on an integer
4  int* pointer_on_a;
5
6  // Store the address in it
7  pointer_on_a = &a;
8
9  // Get the value inside the variable pointed.
10 int b = *pointer_on_a;
```

Example with functions

```
1 void increment_value(int a)
2 {
3     a++;
4 }
5
6 void increment_value_via_pointer(int* a)
7 {
8     (*a)++;
9 }
```

```
1 int a = 123;
2 increment_value(a);
3 // Variable 'a' is still 123.
4
5 increment_value_via_pointer(&a);
6 // Variable 'a' is now 124.
```

Time to practise: 6.Pointers

In this exercise, you will:

- Declare pointers.
- Get the address of a variable.
- Receive pointer arguments in functions.
- Dereference pointers.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Arrays

In C, there are two ways to allocate arrays:

- Static allocation
- Dynamic allocation

Statically allocated arrays

- You know their size at **compile time**.
- They are allocated on the **stack**, and therefore are automatically destroyed when going out-of-scope.

Statically allocated arrays

- To declare an array of 10 integers

```
1 int my_array[10];
```

- To access the 3rd element in this array

```
1 my_array[2]
```

Shouldn't the 3rd element be [3]?

In C, indexes start at 0 so the first element is not [1] but [0], and the last element is never [size] but [size-1].

Dynamically allocated arrays

- Their size is not required until **runtime**.
- They are allocated on the **heap**, where the corresponding memory is not deallocated until explicitly freed by the developer.

Dynamically allocated arrays

To allocate an array, we use the function `malloc`.

```
1 void* malloc(size_t size);
```

- It takes the number of **bytes** to allocate. It is encoded as a `size_t`, which is just a type that can hold “large” integers.
- It is not aware of a **data type**.
- It is not aware of a **number of dimensions**.

Tip: finding a type size

You can find the number of bytes that a data type occupies by calling `sizeof(my_type)`.

Dynamically allocated arrays

- The function `malloc` returns a pointer on the first element in the array allocated.
- If the allocation failed, it returns the value `NULL`.
- The function `malloc` does not know the type of data you want to put in that memory, so it returns a pointer on “anything”, represented with `void*`.
 - It is good practice to cast it back to a pointer on your type.

Example: to allocate an array of 10 integers

```
1 int* my_array = (int*) malloc(sizeof(int) * 10);  
2 my_array[0] = 123;  
3 my_array[1] = 456;
```

Dynamically allocated arrays

- The memory allocated is not deallocated until specified explicitly by the developer using the function `free`.
- The function `free` takes the pointer that was returned by a previous call to `malloc`.

Example: to free a dynamically allocated array of 10 integers

```
1 int* my_array = (int*) malloc(sizeof(int) * 10);  
2 // Some code  
3 free(my_array);
```

Memory leaks

A *memory leak* is when a dynamically allocated memory is not freed, thus wasting memory resources for the entire duration of your program.

Receiving arrays as function arguments

■ As an array

```
1 void my_function(int my_array[10])
2 {
3     // Some code
4     my_array[0] = 123;
5     my_array[1] = 456;
6     // ...
7 }
8
9 int main(int argc, char* argv[])
10 {
11     int array[10];
12     my_function(array);
13 }
```

Receiving arrays as function arguments

■ As a pointer

```
1 void my_function(int* my_array)
2 {
3     // Some code
4     my_array[0] = 123;
5     my_array[1] = 456;
6     // ...
7 }
8
9 int main(int argc, char* argv[])
10 {
11     int* array = (int*) malloc(sizeof(int) * 10);
12     my_function(array);
13     free(array);
14 }
```

Statically allocated arrays - Multidimensional

- To declare a 2D array

```
1 int my_array[row_count][column_count];
```

- To access the element on the 2nd row, 5th column

```
1 my_array[1][4]
```

Multidimensional arrays

- The concept of multidimensional arrays only really exists in your mind's eye, because your computer's memory is just one big 1D array.
- When we allocate memory, whether is on the stack or on the heap, it is contiguous (i.e. there is no hole in it).
- Therefore, consecutive elements are consecutive in memory.
- With multidimensional arrays, programming languages must decide how to lay out dimensions in memory. Consecutive elements in one dimension will be consecutive in memory, while consecutive elements in a different dimension will require jumps in memory.

Multidimensional arrays

- The C language is said to be “row-major order”, it means that elements in the same row are consecutive in memory.
- Regardless of the number of dimensions, this means that elements in the **rightmost dimension** are consecutive in memory.
- The last element of a row, and the first element of the following row are consecutive in memory too. Remember that, in memory, all rows are next to each other, in a big 1D array.

Example:

```
1 int my_2D_array[20][10];  
2 my_2D_array[0][0] = 123;  
3 my_2D_array[0][1] = 456;  
4 my_2D_array[0][2] = 789;  
5 // etc...
```

Passing multidimensional arrays as function arguments

```
1 void my_function(int my_array[20][10])
2 {
3     my_2D_array[0][0] = 123;
4     my_2D_array[0][1] = 456;
5     my_2D_array[0][2] = 789;
6     // etc...
7 }
8
9 int main(int argc, char* argv[])
10 {
11     int array[20][10];
12     my_function(array);
13 }
```

Time to practise: 7.Arrays

In this exercise, you will:

- Allocate arrays statically
- Allocate arrays dynamically
- Deallocate a dynamically allocated array
- Allocate a multidimensional array
- Pass arrays to functions

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Structures

In C, you can create “compound” datatypes using structure, they allow you to group together variables into a coherent unit. For example:

```
1 struct person
2 {
3     float weight;
4     float height;
5 };
```

Structures accessors

To access members of a structure, you need to use the dot '.'.

```
1 struct person
2 {
3     float weight;
4     float height;
5 };
6
7 struct person p;
8 p.weight = weight;
9 p.height = height;
```

Structure pointers

If you have a pointer on a structure, say in a function, you can *dereference* it, then access the members.

```
1 float get_person_height(struct person* p)
2 {
3     return (*p).height;
4 }
```

Structure pointers - Syntactic sugar

This `(*my_struct_pointer).member` notation is a bit cumbersome. So, when using a structure pointer, you can use the arrow `'->'` instead.

```
1 float get_person_height(struct person* p)
2 {
3     return p->height;
4 }
```

Only syntactic sugar

The `"x->y"` notation is only for convenience: it does nothing else than converting to `"(*x).y"`.

Typedef - Technique

- The `struct` keyword in `struct person` can be redundant.
- You can "rename" the `struct person` in `person_t` using `typedef`.
- This is only a syntactic change, the structure itself is not changed.

Convention

When using `typedef`, you can no longer tell that the type is a structure, so we typically append `_t` to the name instead.

Typedef - Example

In the code below, from line 6, struct `person` and `person_t` can be used interchangeably.

```
1 typedef struct person
2 {
3     float weight;
4     float height;
5 } person_t;
6
7 person_t record_person(float weight, float height)
8 {
9     person_t p;
10    p.weight = weight;
11    p.height = height;
12 }
```

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Decoupling function declaration and definition

- So far, we only defined functions, and had to put them above the function `main`.
- A function declaration provides the interface:
 - The **name** of the function.
 - The **return type** of the function.
 - The **list of arguments** to the function.
- A function declaration ends with a semi-colon `';`.

Example

■ A function declaration

```
1 void my_function();
```

■ A function definition

```
1 void my_function()  
2 {  
3     // Some code  
4 }
```

Location of function definitions

If a function declaration is present before the function `main`, the corresponding function definition can be:

- further down in the same file.
- in a different file (as you will learn in the next unit).

Example

```
1 // Function declaration
2 int add_numbers(int a, int b);
3
4 // Main function
5 int main(int argc, char* argv[])
6 {
7     int total = add_numbers(123, 456);
8     printf("Total number is %d.\n", total);
9 }
10
11 // Function definition
12 int add_numbers(int a, int b)
13 {
14     return a + b;
15 }
```

Time to practise: 8.FunctionDeclarations

In this exercise, you will:

- Write a function declaration
- Decouple function declaration and definition.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Source and headers

In C, there are two types of files

Source Contains functions definitions. Uses ".c" file extension.

Header Contains functions declarations. Uses ".h" file extension.

Ideally, you want to split your code such that functions are grouped by theme / semantics.

Splitting the code

File my_functions.h

```
1 void add_numbers(int a, int b);
```

File my_functions.c

```
1 void add_numbers(int a, int b)
2 {
3     return a + b;
4 }
```

File main.c

```
1 int main(int argc, char* argv[])
2 {
3     int total = add_numbers(123, 456);
4     printf("Total number is %d.\n", total);
5 }
```

Including headers

- When your source code is scattered across multiple files, you may call a function that is declared, and defined, in a different file.
- To make sure the compiler knows where to look, you must include the corresponding header.
- To do so, you use the `#include` preprocessing directive.

Example of a multi-file structure

File my_functions.c

```
1 #include "my_functions.h"
2 void add_numbers(int a, int b)
3 {
4     return a + b;
5 }
```

File main.c

```
1 #include "my_functions.h"
2 int main(int argc, char* argv[])
3 {
4     int total = add_numbers(123, 456);
5     printf("Total number is %d.\n", total);
6 }
```

Inclusion guard

- When you include a file, the include line is replaced with the content of the file included before the compilation starts.
- You must make sure that a header cannot be included multiple times, or issues of duplicate declarations can arise.
- To guarantee a single inclusion, you can use *preprocessor directives*.

Preprocessor directives

Definition

Preprocessor directive are instructions given to the preprocessor, which treats the source code before the compiler reads it.

Preprocessor directives have three main uses:

- include other files (c.f.: `#include`)
- exclude certain portions of code from compilation.
- automatically replace certain occurrences with literals.

Preprocessor directives as inclusion guards

```
1 #ifndef SOME_NAME
2     #define SOME_NAME
3     void my_function();
4 #endif
```

- The first time this header is included, the symbol `SOME_NAME` does not exist.
- The `ifndef` directive, which stands for “**if not defined**”, checks that the symbol `SOME_NAME` does not exist. It is the case here indeed, so it executes the block associated.
- The `define` declares the symbol, and the function is declared in the line that follows.
- From now on, including this header will have no effect because the symbol `SOME_NAME` is now defined and the check done by the directive `ifndef` will systematically fail.

Preprocessor directives as occurrence replacement

- Defines can also assign a value to a symbol.
- Each occurrence of that symbol will be replaced with the value provided.
- This is useful to easily change the size of statically allocated arrays passed through functions without having to update all values one by one manually.

Example

- Example: every occurrence of “SIZE” will be replaced in the source code before it is read by the compiler for compilation.

```
1 #define SIZE 100
2 void my_function(int my_array[SIZE])
3 {
4     my_array[0] = 123;
5     my_array[1] = 456;
6     // etc...
7 }
8
9 int main(int argc, char* argv[])
10 {
11     int array[SIZE];
12     my_function(array);
13 }
```

Example

- What the compiler receives, once the preprocessor finished its part.

```
1 void my_function(int my_array[100])
2 {
3     my_array[0] = 123;
4     my_array[1] = 456;
5     // etc...
6 }
7
8 int main(int argc, char* argv[])
9 {
10     int array[100];
11     my_function(array);
12 }
```

Multi-file compilation

You now have headers with function declarations, source files with function definitions and your main file. However, you must still compile it all. To do so:

- You must list all sources files.
- You must **not** list header files.
- Assuming you have two source files: `main.c` and `my_functions.c`, and one header `my_functions.h`:

```
1 gcc -o main main.c my_functions.c
```

Time to practise: 9.MultiFileStructure

In this exercise, you will:

- Place a function declaration in a different file.
- Place a function definition in a different file.
- Cover all includes needed.
- Compile everything, and execute.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Pagerank

- Now that you have acquired strong foundations of C programming, here is a little challenge.
- Developing a program known as **PageRank**.
- It was originally developed to sort webpages based on popularity, and became the backbone of the Google Search Engine.

Graphs

- Pagerank uses the graph data structure to represent the net.
- A graph is a series of vertices connected with edges, like cities linked with roads.
- In Pagerank, every vertex represents a webpage, and every edge represents a hyperlink from a given webpage to another webpage.
- The above repeats until a termination condition, in this case a maximum number of iterations, is reached.

Algorithm

- Every vertex begins with a value of $\frac{1}{n}$, with n equal to the number of vertices.
- At every iteration, each vertex splits its value equally across all of its neighbours, and adds it to the neighbour's existing value.
- Each neighbour multiplies its value by $(1 - \alpha)$, and then adds α .
- α is the *dump factor*: it is used to control how much of the webpage popularity comes from the number of links pointing to it.

Time to practice: 10.Pagerank

You will find guidelines to get you to develop the Pagerank application.

- Take your time.
- Do it step by step.
- Do not hesitate to ask questions.

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Naming conventions

- In C, we tend to use exclusively lowercase letters, using underscores when needed.

```
1 int here_is_an_int = 123;
```

- For constants, we tend to use exclusively uppercase letters, using underscores when needed.

```
1 const int HERE_IS_A_CONST_INT = 123;
```

Indentation conventions

- Certain programming languages such as Python enforce a specific indentation.
- In C, you are free to indent it the way you want.
- Here are two of the most popular styles, pick whichever you prefer, but **stay consistent**.

```
1  if(condition) {  
2      // Some code  
3  }
```

```
1  if(condition)  
2  {  
3      // Some code  
4  }
```

Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Comments

- Single-line comments start with `//`.

```
1 // Here is a single-line comment
```

- Multi-line comments are surrounded with `/*` and `*/`.

```
1 /* Here is a  
2    multi-line comment */
```

- Documentation comments are surrounded with `/**` and `*/`⁴.

```
1 /** Here is a  
2    documentation comment */
```

⁴or `*/` if you want to make it symmetrical, though this is pure aesthetic.

Documentation

- Learn [Doxygen](#).
- Simple markup to insert in your comments.
- Run a command and it will automatically generate an entire website, and PDF if you want.
- Can make a good impression in interviews...

```
1  /**
2   * @brief This function calculates the quotient
3   * between the two numbers provided.
4   * @param[in] a The dividend.
5   * @param[in] b The divisor.
6   * @return The quotient of the division.
7   * @pre \param b must not be equal to 0.
8   */
9  float divide(float a, float b);
```


Table of Contents

1 Introduction

2 Unit 1: Variables

3 Unit 2: Operators

4 Unit 3: Branching

5 Unit 4: Loops

6 Unit 5: Functions

7 Unit 6: Pointers

8 Unit 7: Arrays

9 Unit 8: Structures

10 Unit 9: Function declarations

11 Unit 10: Multi-file structure

12 Unit 11: Application

13 Good practice

14 Meta-programming

15 Summary

Summary

- **Congratulations:** you have learned the essentials of the C programming language and compilation.
- Feel free to refer back to this coding refresher at any point in your studies.
- Admittedly, there are also more advanced concepts in C. If you are curious, feel free to explore the web or ask your teachers to learn more.